

FEN – Fast Earn Nearby

Master Build-Proof Product + Technical Handoff Pack

Version 1.0 | Combined source of truth for MVP build, product rules, schema, security, monetisation, and implementation order

Combined source of truth for product rules, build decisions, schema, security, monetisation, implementation order, acceptance criteria, and coding instructions.

1. Purpose of this document

This document combines the product strategy, build decisions form, developer handoff, and technical pack into one master file. A coding AI or developer should be able to use this as the single source of truth for the FEN MVP without needing additional clarification for normal implementation decisions. The goal is to keep the MVP small, low-cost, understandable, and ready for later scale-up when finances improve.

2. Product summary

- Product name: FEN – Fast Earn Nearby
- Core idea: a local platform where people can quickly post small jobs, find nearby help, or earn money from simple local work
- Primary audience: local residents, including communities with many older users and less tech-confident users
- MVP objective: prove local demand and usability with the lightest operational and financial overhead possible
- Primary trust principle: clear actions, simple language, limited public data exposure, and manual moderation in early stages

3. Goals and non-goals

Area	Included in MVP	Excluded from MVP
Matching	Poster posts job, workers apply, poster selects one	Instant one-click assignment without applications
Location	Postcode-based local matching	Precise maps and exact public addresses
Messaging	Text chat after selection only	Public pre-match chat, image/file chat
Monetisation	Free tier rule ready from day one; paid platforming structurally	Complex in-app commissions in MVP
Trust	Verification, reports, blocks, manual moderation	Early review system, identity verification, automated scoring
Payments	None in MVP	In-app payments or payout rails

4. Zero-budget launch strategy and scale-up path

- Use Expo managed workflow to keep mobile builds simple and cheap.
- Use Supabase free tier initially for auth, database, and realtime.
- Avoid in-app payments in MVP to reduce legal, support, and engineering overhead.
- Use postcode-based distance approximation instead of expensive geospatial systems in MVP.
- Moderate manually via founder/admin workflow rather than building a full moderation back office immediately.
- Design the schema so subscriptions, boosts, reviews, trust scoring, and geocoding can be added later without rebuilding the app structure.

5. Locked build decisions

Decision area	Locked choice
Project name	FEN – Fast Earn Nearby
MVP version	FEN MVP v1
Primary target	Android first, but codebase supports both Android and iPhone
Codebase	React Native + Expo Router + Supabase
Language	TypeScript
Matching model	Workers apply, poster selects one
Can one account both post and work	Yes
Can guests browse before login	Yes
Reviews in MVP	No
AI suggestions in MVP	Yes, optional only
Dark mode	Yes
Location method	Postcode only in MVP

6. User types and responsibilities

Role	Core actions
Guest	Browse public-safe job listings only
Verified standard user	Post jobs, apply to jobs, message after selection, manage own profile
Poster	Create jobs, review applicants, select worker, cancel, reopen if eligible, mark complete
Worker	Browse, apply, message if selected, cancel if necessary, mark complete
Founder/Admin	Review reports, hide jobs, suspend users, inspect operational data, handle manual moderation

7. Authentication and account lifecycle

Area	Decision / rule
Login method	Email/password
Email verification required	Yes for posting, applying, and messaging
Can unverified users browse	Yes
Can unverified users post	No
Can unverified users apply	No
Can unverified users message	No
Password reset	Email reset flow via Supabase
Account deletion	Soft delete / anonymise
Suspended account behaviour	Cannot use normal product flows; founder decides whether existing content is hidden or archived

8. Profiles

Field	Required	Notes
display_name	Yes	Shown publicly
email	Yes	Mirrors auth record for convenience
postcode	Yes	Used for local matching
bio	No	Optional free text
skills	No	Optional free text in MVP
transport_mode	No	Single-select enum
profile_photo_url	No	Optional
preferred_role	No	Preference only, not hard permission split
plan_tier	System	free or worker_plus
completed_jobs_count	System	Basic trust indicator
is_suspended	System	Admin-controlled

9. Job posting form specification

Field	Rule
Title	Required, max 80 chars
Description	Required, max 1000 chars
Budget	Required, fixed amount in GBP for MVP
Postcode	Required
Urgency	Required: Need now / Today / Flexible
Preferred date/time	Optional
Category	Optional or AI-suggested
Photos	Deferred from strict build-proof MVP
Tools supplied flag	Optional, default false

10. Banned job categories

- Illegal activity
- Adult services
- Hazardous work without proper safeguards
- Money transfer or scam-style tasks
- Violence, drugs, weapons, or theft-related requests
- Regulated work requiring licensing where this cannot be assured

11. Browse and discovery

Decision	Rule
Default sort	Mixed weighting: newest and approximate distance
Filters	Category, urgency, budget range, distance radius
Search	Yes, title/description keyword search
Radius options	1, 5, 10, 25 miles
Distance shown	Approximate only
Exact public location	Never shown before selection
Public listing location	Area or postcode district only

12. Matching flow

The MVP uses an application model rather than instant assignment. This creates more control for the poster and is safer for trust and local logistics.

- Poster creates job
- Workers apply
- Poster reviews applications
- Poster selects one worker
- Messaging unlocks for poster and selected worker only
- Other applicants are automatically rejected

13. Messaging rules

Rule area	Decision
Messaging available before match	No
Chat type	Text only
Links allowed	Yes, but abuse can be manually moderated
Delete messages	No
Edit messages	No
After cancellation	Conversation archived/read-only
After completion	Conversation archived/read-only in strict MVP
Blocked users and old messages	Blocked users should not continue normal message access through UI

14. Status lifecycle

Status	Meaning
open	Job is visible and accepting applications
held	Poster selected worker but timing/details not confirmed
confirm_pending	Start time agreed
in_progress	Work has started
completed	Work finished
cancelled	Ended before completion

15. Allowed status transitions

From	To	Who can trigger	Notes
open	held	Poster	Selects one worker
held	confirm_pending	Poster or selected worker	Timing agreed and stored
confirm_pending	in_progress	Poster or selected worker	Work begins
in_progress	completed	Poster or selected worker	Either side can complete in MVP
open / held / confirm_pending	cancelled	Poster, selected worker, or admin	Reason required
cancelled	open	Poster	Only if work never started

16. Anti-ghosting and cancellation rules

- Confirmed timing means a specific agreed date/time is stored on the job.
- Protected cancellation window begins once status becomes confirm_pending.
- Cancellation reason is required.
- MVP stores cancellation data but does not yet apply visible trust penalties.
- Warning copy should tell users cancelling after timing is agreed may inconvenience the other person.

17. Trust, safety, reports, and blocking

Capability	MVP decision
Public reviews	No
Basic trust indicators	Verified email, account age, completed jobs count
Reports allowed on	User, job, message
Blocking feature	Yes
Moderation	Manual founder/admin moderation first
Auto-hide	Not required in MVP; manual action preferred
Audit requirement	Admin actions should record who acted, on what, and when

18. Location and privacy rules

- Location method in MVP is postcode only.
- Distance is approximate and based on postcode logic, not exact routing.
- Exact public address is not stored for browse presentation and must not be shown on public cards.
- Public cards should only show area or postcode district.
- Exact address or fine-grained meeting details should be shared privately after selection and only when needed.

19. SQL schema baseline

```
create extension if not exists pgcrypto;

create type public.job_status as enum ('open','held','confirm_pending','in_progress','completed','cancelled');
create type public.application_status as enum ('applied','rejected','selected','withdrawn');
create type public.report_status as enum ('open','reviewing','resolved');
create type public.transport_mode as enum ('walk','cycle','drive','public_transport','unspecified');
create type public.plan_tier as enum ('free','worker_plus');

create table public.profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  email text,
  display_name text not null,
  postcode text not null,
  bio text,
  skills text,
  transport_mode public.transport_mode not null default 'unspecified',
  profile_photo_url text,
  preferred_role text,
  plan_tier public.plan_tier not null default 'free',
  application_limit_override integer,
  completed_jobs_count integer not null default 0,
  is_suspended boolean not null default false,
  deleted_at timestamptz,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table public.jobs (
  id uuid primary key default gen_random_uuid(),
  poster_id uuid not null references public.profiles(id) on delete cascade,
  title text not null,
  description text not null,
  budget_gbp numeric(10,2) not null check (budget_gbp >= 0),
  postcode text not null,
  postcode_district text,
  category text,
  urgency text not null check (urgency in ('Need now','Today','Flexible')),
  preferred_start_at timestamptz,
  agreed_start_at timestamptz,
  status public.job_status not null default 'open',
  accepted_worker_id uuid references public.profiles(id) on delete set null,
  tools_supplied boolean not null default false,
  is_reopened boolean not null default false,
  cancel_reason text,
  deleted_at timestamptz,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table public.applications (
  id uuid primary key default gen_random_uuid(),
  job_id uuid not null references public.jobs(id) on delete cascade,
  worker_id uuid not null references public.profiles(id) on delete cascade,
  message text,
  status public.application_status not null default 'applied',
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  constraint applications_unique_worker_per_job unique (job_id, worker_id)
);

create table public.conversations (
  id uuid primary key default gen_random_uuid(),
  job_id uuid not null references public.jobs(id) on delete cascade,
  poster_id uuid not null references public.profiles(id) on delete cascade,
  worker_id uuid not null references public.profiles(id) on delete cascade,
  is_archived boolean not null default false,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  constraint conversations_unique_job_worker unique (job_id, worker_id)
);

create table public.messages (
  id uuid primary key default gen_random_uuid(),
  conversation_id uuid not null references public.conversations(id) on delete cascade,
  job_id uuid not null references public.jobs(id) on delete cascade,
  sender_id uuid not null references public.profiles(id) on delete cascade,
  body text not null,
  created_at timestamptz not null default now()
);

create table public.reports (
  id uuid primary key default gen_random_uuid(),
  reporter_id uuid not null references public.profiles(id) on delete cascade,
  reported_user_id uuid references public.profiles(id) on delete set null,
  job_id uuid references public.jobs(id) on delete set null,
  message_id uuid references public.messages(id) on delete set null,
```

```

reason text not null,
details text,
status public.report_status not null default 'open',
admin_note text,
created_at timestamptz not null default now(),
updated_at timestamptz not null default now()
);

create table public.blocks (
  id uuid primary key default gen_random_uuid(),
  blocker_id uuid not null references public.profiles(id) on delete cascade,
  blocked_id uuid not null references public.profiles(id) on delete cascade,
  created_at timestamptz not null default now(),
  constraint blocks_unique_pair unique (blocker_id, blocked_id)
);

create table public.subscriptions (
  id uuid primary key default gen_random_uuid(),
  profile_id uuid not null references public.profiles(id) on delete cascade,
  tier public.plan_tier not null,
  starts_at timestamptz not null,
  ends_at timestamptz,
  is_active boolean not null default true,
  created_at timestamptz not null default now()
);

create index jobs_status_created_idx on public.jobs(status, created_at desc);
create index jobs_postcode_district_idx on public.jobs(postcode_district);
create index jobs_poster_idx on public.jobs(poster_id);
create index jobs_worker_idx on public.jobs(accepted_worker_id);
create index applications_job_status_idx on public.applications(job_id, status);
create index applications_worker_created_idx on public.applications(worker_id, created_at desc);
create index messages_conversation_created_idx on public.messages(conversation_id, created_at);

```


20. RLS source-of-truth baseline

```
alter table public.profiles enable row level security;
alter table public.jobs enable row level security;
alter table public.applications enable row level security;
alter table public.conversations enable row level security;
alter table public.messages enable row level security;
alter table public.reports enable row level security;
alter table public.blocks enable row level security;
alter table public.subscriptions enable row level security;

create policy profiles_select_self on public.profiles
for select to authenticated
using (auth.uid() = id);

create policy profiles_update_self on public.profiles
for update to authenticated
using (auth.uid() = id)
with check (auth.uid() = id);

create policy profiles_insert_self on public.profiles
for insert to authenticated
with check (auth.uid() = id);

create policy jobs_public_read_open on public.jobs
for select
using (deleted_at is null);

create policy jobs_insert_own on public.jobs
for insert to authenticated
with check (auth.uid() = poster_id);

create policy jobs_update_poster on public.jobs
for update to authenticated
using (auth.uid() = poster_id)
with check (auth.uid() = poster_id);

create policy applications_insert_worker on public.applications
for insert to authenticated
with check (
    auth.uid() = worker_id
    and exists (
        select 1 from public.jobs j
        where j.id = job_id
        and j.poster_id <> auth.uid()
        and j.status = 'open'
        and j.deleted_at is null
    )
);

create policy applications_read_worker_or_poster on public.applications
for select to authenticated
using (
    auth.uid() = worker_id
    or exists (
        select 1 from public.jobs j
        where j.id = job_id
        and j.poster_id = auth.uid()
    )
);

create policy conversations_read_participants on public.conversations
for select to authenticated
using (auth.uid() = poster_id or auth.uid() = worker_id);

create policy messages_read_participants on public.messages
for select to authenticated
using (
    exists (
        select 1 from public.conversations c
        where c.id = conversation_id
        and (c.poster_id = auth.uid() or c.worker_id = auth.uid())
    )
);

create policy messages_insert_participants on public.messages
for insert to authenticated
with check (
    auth.uid() = sender_id
    and exists (
        select 1
        from public.conversations c
        where c.id = conversation_id
        and (c.poster_id = auth.uid() or c.worker_id = auth.uid())
        and c.is_archived = false
    )
);

create policy reports_insert_authenticated on public.reports
```

```
for insert to authenticated
with check (auth.uid() = reporter_id);

create policy reports_read_reporter on public.reports
for select to authenticated
using (auth.uid() = reporter_id);

create policy blocks_manage_self on public.blocks
for all to authenticated
using (auth.uid() = blocker_id)
with check (auth.uid() = blocker_id);

create policy subscriptions_read_self on public.subscriptions
for select to authenticated
using (auth.uid() = profile_id);
```

21. Server-side logic / RPC baseline

```
create or replace function public.can_apply_to_job(p_job_id uuid)
returns boolean
language plpgsql
security definer
as $$
declare
    v_uid uuid := auth.uid();
    v_plan public.plan_tier;
    v_override integer;
    v_count integer;
    v_job_open boolean;
begin
    if v_uid is null then
        return false;
    end if;

    select plan_tier, application_limit_override
    into v_plan, v_override
    from public.profiles
    where id = v_uid and deleted_at is null and is_suspended = false;

    if not found then
        return false;
    end if;

    select exists(
        select 1 from public.jobs
        where id = p_job_id
            and status = 'open'
            and poster_id <> v_uid
            and deleted_at is null
    ) into v_job_open;

    if not v_job_open then
        return false;
    end if;

    if v_plan = 'worker_plus' then
        return true;
    end if;

    select count(*) into v_count
    from public.applications
    where worker_id = v_uid
        and created_at >= now() - interval '7 days';

    if v_override is not null then
        return v_count < v_override;
    end if;

    return v_count < 3;
end;
$$;

create or replace function public.select_worker_for_job(p_job_id uuid, p_worker_id uuid)
returns void
language plpgsql
security definer
as $$
declare
    v_uid uuid := auth.uid();
begin
    update public.jobs
    set accepted_worker_id = p_worker_id,
        status = 'held',
        updated_at = now()
    where id = p_job_id
        and poster_id = v_uid
        and status = 'open';

    if not found then
        raise exception 'Unable to select worker';
    end if;

    update public.applications
    set status = case when worker_id = p_worker_id then 'selected' else 'rejected' end,
        updated_at = now()
    where job_id = p_job_id;

    insert into public.conversations (job_id, poster_id, worker_id)
    select j.id, j.poster_id, p_worker_id
    from public.jobs j
    where j.id = p_job_id
    on conflict (job_id, worker_id) do nothing;
end;
$$;
```

22. Monetisation model

Plan / feature	Rule / illustrative price	Status
Free worker tier	Maximum 3 applications per rolling 7 days	MVP-ready
Worker Plus	Illustrative £2.99–£4.99/month for unlimited applications	Dependencies: user activation
Job Boost	Illustrative £1–£3 per listing	Later
Featured urgent slot	Illustrative £2–£5	Later

23. Screen map and route structure

```
app/  
  (auth)/  
    sign-in.tsx  
    sign-up.tsx  
    forgot-password.tsx  
  
  (app)/  
    browse.tsx  
    post.tsx  
    my-jobs.tsx  
    messages.tsx  
    profile.tsx  
    job/[id].tsx  
    applications/[jobId].tsx  
  
components/  
  jobs/JobCard.tsx  
  jobs/JobFilters.tsx  
  jobs/StatusChip.tsx  
  chat/ConversationListItem.tsx  
  ui/AppButton.tsx  
  ui/AppTextField.tsx  
  
lib/  
  supabase.ts  
  auth.ts  
  jobs.ts  
  applications.ts  
  messaging.ts  
  monetisation.ts  
  distance.ts  
  
types/  
  database.ts  
  jobs.ts  
  profile.ts
```

24. Screen responsibilities

Screen	Primary purpose	Critical actions
Browse	Discover jobs	Search, filter, open job detail
Post	Create a job	Validate and submit
Job Detail	View one job	Apply if eligible
My Jobs	Track poster and worker activity	View status, open applicants, reopen/cancel
Applications	Poster reviews applicants	Select worker
Messages	Conversation list and chat	Read/send text messages
Profile	Manage account	Edit details, view plan, logout

25. UI and copy rules

- Primary colour family: violet / purple.
- Dark mode is part of MVP.
- Use plain, warm, direct language.
- Avoid jargon such as gig, hustle, bid, escrow, marketplace where possible.
- Prioritise large text, strong contrast, and generous tap targets.
- Keep cards spacious and avoid dense layouts.

26. AI features and disclaimers

AI feature	MVP or later	Rule
Category suggestion	MVP	Optional helper only
Budget suggestion	MVP	Optional helper only
Wording improvement	MVP	Optional helper only
Summary generation	Later	Not needed in MVP
Moderation assistance	Later	Manual moderation first

27. Required AI disclaimer

AI suggestions are guidance only. Final job details, pricing, and arrangements are chosen by the users.

28. Acceptance criteria

- A user can register, verify email, and sign in.
- A verified user can create a job successfully.
- A verified worker can browse nearby jobs and apply.
- The 3-applications-per-7-days rule works correctly for free users.
- A poster can view applicants and select one worker.
- Only poster and selected worker can message each other.
- Jobs move through valid statuses only.
- Users can report jobs, users, and messages.
- Founder/admin can hide unsafe content and suspend users.
- Public browse results do not reveal exact address.

29. Mandatory test cases

- Happy path: poster creates job, worker applies, poster selects worker, both message, job completes.
- Worker hits free application cap and is blocked from 4th application in rolling 7-day window.
- Duplicate application attempt fails.
- Unverified user can browse but cannot post, apply, or message.
- Suspended user is blocked from normal use.
- Cancellation after timing confirmation requires reason.
- Poster cannot select two workers for one job.
- Blocked users do not continue normal chat access.

30. Implementation order

- Create enum types, tables, indexes, and updated_at handling.
- Enable RLS and apply policies.
- Build auth and automatic profile creation.
- Build browse and job detail screens.
- Build post job form and validation.
- Build application flow and weekly cap enforcement.
- Build applicant review and worker selection.
- Build conversations and chat.
- Build status transitions, cancellation, and reopen logic.
- Build reports and blocks.
- Add optional AI helpers only after core flow works.

31. Non-negotiable coding rules

- Do not move monetisation checks to client-only logic.
- Do not allow public messaging before worker selection.
- Do not reveal exact address in public browse results.
- Do not add in-app payments to MVP.
- Do not invent extra statuses or major business rules without approval.
- Do not make AI a dependency for posting or browsing.

32. Instructions to the coding AI

Use this document as the source of truth for the FEN MVP.
Do not invent business rules.
If a high-impact implementation detail is ambiguous, ask before coding.
Build MVP only.
Use React Native + Expo Router + Supabase unless explicitly approved otherwise.
Generate the database schema first.
Then generate RLS policies.
Then generate the route structure and screen list.
Then build file by file.
Keep the code easy to review and low-maintenance.
Prefer low-cost implementation choices.